CURSO UNIVERSITARIO

SW505 - Construcción de Software I

Semana 3 — Anticipación al cambio, estándares de codificación y fundamentos de construcción asistida por IA

M. Sc. Juan Luis Herencia Guerra

Unidad 1 · Fundamentos de la construcción de software

CONTENIDO DE LA SESIÓN

Agenda

01

Diseño para el cambio

Puntos de variación y mecanismos de extensión (polimorfismo, DI, configuración, Strategy).

02

Reutilización y estándares

Niveles de reutilización, nomenclatura, formato, comentarios y organización de archivos.

03

Construcción asistida por IA

Prompting como especificación, descomposición de tareas, riesgos y responsabilidad profesional.

04

Documentación y cierre

Documentación técnica mínima y síntesis de la semana.

BLOQUE 1

Diseño para el cambio



Puntos de variación y mecanismos que aíslan el costo del cambio dentro de una frontera controlada.

BLOQUE 1 · DISEÑO PARA EL CAMBIO

El cambio como inevitable y anticipable

Todo sistema de software cambia: los requisitos evolucionan, el entorno tecnológico se transforma, el negocio se adapta.

✓ **La pregunta correcta**	No es si el sistema cambiará, sino cuándo y dónde.
✓ **Aislamiento del cambio**	El constructor que anticipa el cambio diseña mecanismos de aislamiento que evitan que el costo de modificar una parte se propague al resto del sistema.
✓ **Punto de variación**	Es una decisión de diseño que se sabe que puede cambiar. Encapsularlo significa que el cambio afecta solo al código dentro de esa frontera.

Cuatro mecanismos de extensión

Formas concretas de encapsular un punto de variación sin que el resto del sistema conozca los detalles.

- Polimorfismo — el comportamiento variable se expresa como implementaciones de una interfaz común.
- Inyección de dependencias — los colaboradores de un módulo se suministran desde fuera, en lugar de instanciarlos internamente.
- Parámetros de configuración — los valores que cambian entre entornos se externalizan, no se incrustan en el código.
- Patrón Strategy — el algoritmo variable se encapsula en un objeto intercambiable, independiente del contexto que lo invoca.

BLOQUE 1 · MECANISMO 1

Polimorfismo

El comportamiento variable se expresa como implementaciones de una interfaz común.

- ✓ El código cliente (pagar) no sabe qué implementación usa.
- ✓ Agregar un nuevo método de pago no requiere modificar el código existente.
- ✓ Reduce el acoplamiento entre el flujo de negocio y los detalles de cada método.

JAVA

```
interface MetodoPago {
  void procesar(double monto);
}

class PagoTarjeta implements MetodoPago {
  public void procesar(double monto) {
    // lógica específica de tarjeta
  }
}

class PagoYape implements MetodoPago {
  public void procesar(double monto) {
    // lógica específica de Yape
  }
}

void pagar(MetodoPago metodo, double monto) {
  metodo.procesar(monto);
}
```

El cliente pagar(...) permanece estable aunque se agreguen nuevos métodos de pago.

BLOQUE 1 · MECANISMO 2

Inyección de dependencias

Los colaboradores de un módulo se suministran desde fuera, en lugar de instanciarlos internamente.

- ✓ Facilita el reemplazo del colaborador sin tocar la clase que lo usa.
- ✓ Facilita la prueba: se puede inyectar un doble de prueba en lugar del servicio real.
- ✓ El objeto Pedido no decide con qué notificador trabaja.

PYTHON

```python
class ServicioNotificacion:
    def enviar(self, mensaje: str) -> None:
        ...

class Pedido:
    def __init__(self, notificador: ServicioNotificacion):
        self._notificador = notificador

    def confirmar(self):
        self._notificador.enviar(
            "Pedido confirmado"
        )

# se inyecta desde fuera
pedido = Pedido(EmailNotificador())
```

Cambiar de EmailNotificador a SmsNotificador no requiere tocar la clase Pedido.

BLOQUE 1 · MECANISMO 3

Parámetros de configuración

Los valores que cambian entre entornos —URLs, umbrales, nombres de archivo— se externalizan a archivos de configuración; no se incrustan en el código.

✓ **Por qué importa**	Cambiar de entorno (desarrollo, pruebas, producción) no debería requerir recompilar ni modificar el código fuente.
✓ **Dónde vive el valor**	Archivos .env, variables de entorno, servicios de configuración centralizados (por ejemplo, un gestor de secretos).
✓ **Riesgo si se ignora**	Valores incrustados en el código («hardcodeados») obligan a un despliegue nuevo por cada cambio de umbral o URL, y facilitan filtrar credenciales en el repositorio.

BLOQUE 1 · MECANISMO 4

Patrón Strategy

El algoritmo variable se encapsula en un objeto intercambiable, independiente del contexto que lo invoca.

- ✓ Carrito no implementa la lógica de descuento; la delega.
- ✓ Se pueden agregar nuevas estrategias sin modificar Carrito.
- ✓ Distinto del polimorfismo simple: aquí el objeto completo representa un algoritmo, no una variante de comportamiento de una entidad de negocio.

JAVA

```
interface EstrategiaDescuento {
  double aplicar(double precio);
}

class DescuentoNavidad
    implements EstrategiaDescuento {
  public double aplicar(double precio) {
    return precio * 0.85;
  }
}

class Carrito {
  private EstrategiaDescuento estrategia;

  double total(double precio) {
    return estrategia.aplicar(precio);
  }
}
```

La estrategia se inyecta en Carrito (a menudo combinando Strategy con inyección de dependencias).

BLOQUE 1 · SÍNTESIS

Comparativa de los cuatro mecanismos

Mecanismo	Qué encapsula	Cuándo usarlo
Polimorfismo	Variantes de un mismo comportamiento de negocio	Múltiples implementaciones de una misma operación (pagos, formatos de salida)
Inyección de dependencias	El colaborador concreto que usa un módulo	Siempre que un módulo dependa de un servicio externo o reemplazable
Parámetros de configuración	Valores que cambian según el entorno	URLs, umbrales, credenciales, nombres de archivo
Patrón Strategy	Un algoritmo completo e intercambiable	Reglas de negocio que cambian con frecuencia (descuentos, tarifas, validaciones)

BLOQUE 2

Reutilización y estándares de codificación



De copiar y pegar a librerías versionadas; de la convención tácita al estándar acordado por el equipo.

Niveles de reutilización de componentes

La reutilización reduce defectos porque el código reutilizado ya fue probado en contextos anteriores.

Copiar y pegar	El nivel más bajo y más riesgoso: cada copia debe actualizarse por separado cuando la lógica cambia.
Patrones de diseño	Soluciones reutilizables a nivel de estructura, no de código — como Strategy, visto en el bloque anterior.
Librerías versionadas	El nivel más alto: componentes empaquetados, con control de versiones explícito y ciclo de pruebas propio.

Qué es un estándar de codificación

Un conjunto de reglas acordadas por el equipo que determinan cómo debe escribirse el código.

Propósito	No es estético: reduce la carga cognitiva al leer código escrito por otros —incluido código generado por una IA— y elimina ambigüedades que puedan inducir errores.
Alcance	Cubre nomenclatura, formato, comentarios y organización de archivos — los cuatro ejes que se detallan a continuación.
Quién lo aplica	Todo el equipo, de forma consistente; hoy gran parte se automatiza mediante linters y formateadores.

Nomenclatura significativa

Los nombres deben revelar la intención.

- Evitar abreviaturas crípticas y nombres de una sola letra (salvo en bucles triviales).
- Los nombres de clase deben ser sustantivos; los de función, verbos o frases verbales.
- Usar nombres pronunciables: facilitan la comunicación oral en el equipo.
- Usar nombres buscables: los números mágicos y los nombres de una letra no se pueden buscar en el código.

BLOQUE 2 · FORMATO

Convenciones de formato y automatización

Indentación, longitud de línea, posición de llaves: la consistencia reduce el ruido visual al leer.

Antes	Estas reglas se discutían manualmente en cada revisión de código, generando fricción y decisiones inconsistentes entre proyectos.
Hoy	Se delegan a formateadores automáticos que se ejecutan en cada guardado o en el pipeline: Prettier (JS/TS), Black (Python), gofmt (Go).
Beneficio	Elimina la discusión de estilo en la revisión de código, dejando esa energía para discutir corrección y diseño.

Comentarios: el porqué, no el qué

EVITAR

```
// incrementa i en 1
i++;

// llama a la función
procesarLote(datos);
```

PREFERIR

```
// avanzamos al siguiente lote:
// la API pagina de a 50 registros
i++;

// se reintenta 3 veces porque el
// proveedor externo es inestable
procesarLote(datos);
```

El código debe ser autoexplicativo. Un comentario que reitera el código es ruido; uno que explica una decisión no obvia es información.

BLOQUE 2 · ORGANIZACIÓN

Organización de archivos

Dónde viven las clases, cómo se nombran los paquetes o módulos, cómo se estructura el proyecto.

Convención sobre configuración	Estructuras predecibles (por ejemplo, separar src/, test/, config/) permiten que cualquier persona del equipo encuentre un archivo sin preguntar.
Nombres de paquetes y módulos	Deben reflejar el dominio del problema, no detalles de implementación.
Coherencia con el estándar del equipo	La organización de archivos es tan parte del estándar de codificación como la nomenclatura o el formato.